

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 711 — Applied Cryptography

The primitives work. Using them correctly is the whole problem

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 9 (with CSCE 701 and CSCE 713)

Prerequisites: CSCE 629 Analysis of Algorithms for the complexity arguments and CSCE 658 Randomized Algorithms for the probabilistic reasoning; CSCE 701 runs in parallel and supplies the operational context

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A reviewed cryptographic design for a real requirement — the primitives chosen with justification, the key lifecycle specified, the failure modes named, and the properties you are *not* providing stated explicitly

Course description

This course is about using cryptography correctly, not about breaking it. The primitives are the part of security that works — AES and SHA-2 have resisted decades of concentrated academic attack — and nearly every real cryptographic failure is a misuse: a reused nonce, a missing authentication tag, an unvalidated certificate, a key in a repository. So the course is organised around correct construction and the specific ways it goes wrong.

The governing rule is stated in Module 01 and never relaxed: do not design your own primitives, and do not assemble your own protocols from primitives. Use a reviewed high-level library. This is not timidity — it is the consistent lesson of every published break of a deployed system, and the course teaches you enough to understand *why* the rule holds, which is what lets you apply it with judgement rather than as superstition.

The second theme is that confidentiality is the easy half and is rarely what you actually needed. Integrity, authentication, and freshness are what most systems get wrong — encryption without authentication is a vulnerability rather than a protection, which Module 04 establishes concretely and which the AEAD construction exists to prevent.

The third is that key management is the real subject. A perfect cipher with a key in a Git repository provides nothing (CSCE 701 Module 07), and the questions of where keys live, who can read them, how they rotate, and what happens when one is exposed are harder and more consequential than any algorithm choice. Modules 09 and 10 are about this.

And the closing position is about honest claims. Module 12 covers the post-quantum transition without overstating its urgency, and Module 13 is about stating precisely which properties a construction provides and under which assumptions — because 'it is encrypted' is not a security property, and naming the adversary, the property, and the assumption is.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State what each primitive provides and what it does not.
- 2 Explain why randomness is the foundation and how it fails.
- 3 Choose and use symmetric encryption correctly.
- 4 Explain why authenticated encryption is the only acceptable default.
- 5 Use hash functions for the right purposes and not the wrong ones.
- 6 Explain key exchange and what public-key cryptography is for.
- 7 Explain signatures, certificates, and the trust problem they do not solve.
- 8 Read a TLS handshake and say what each step establishes.
- 9 Store passwords and derive keys correctly.
- 10 Design envelope encryption and a key lifecycle.
- 11 Identify the implementation hazards that break correct designs.
- 12 Assess the post-quantum transition proportionately.
- 13 State a cryptographic claim honestly and completely.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What Cryptography Provides	What problem does cryptography actually solve?
2	Randomness	Where do keys come from, and what if the answer is wrong?
3	Symmetric Encryption	How do you turn a block cipher into something usable?
4	Authenticated Encryption	Why is encryption without authentication a vulnerability?
5	Hash Functions	What property do you actually need from a hash?
6	Key Exchange and Public Keys	How do two parties who have never met agree on a key?

Wk	Module	Core question it answers
7	Signatures and Certificates	A signature proves a key signed it. Now what?
8	TLS, as the Worked Example	What does a handshake actually establish, step by step?
9	Passwords and Key Derivation	Why is a fast hash the wrong tool here?
10	Key Management	Where does the key live, and who can read it?
11	Implementation Hazards	The algorithm is right. Why is the system broken?
12	Post-Quantum Cryptography	What actually breaks, and when should you act?
13	Claiming Cryptographic Security Honestly	What does “it is encrypted” actually tell anyone?

Projects

Project 1 — A primitive, implemented and validated (after Module 7)

Implement one primitive from its specification, validate it against published test vectors, and then write the argument for never using it.

Requirements

- **One primitive implemented from its specification** — ChaCha20, SHA-256, or HMAC. **From the spec, not from a reference implementation.**
- **Validated against the official published test vectors**, with the validation code included and every vector passing.
- **A written account of every specification detail you got wrong first**, and how the vectors caught it.
- **A timing analysis**: identify any place your implementation branches or indexes on secret data (Module 11 §1).
- **And the argument for not using your own implementation**, written in your own words and specific to what you found.

Done means

- **Every official test vector passes** — which is binary, and which is the point of the exercise.
- **The specification mistakes documented honestly**, including the ones that round-tripped correctly and were still wrong. **This is the most instructive part and it is graded.**
- **At least one timing hazard identified in your own code**, because there will be one.
- **And the never-use-this argument specific rather than generic**: what exactly would have to be true for your implementation to be safe, and why it is not.

Project 2 — A cryptographic design, reviewed (after Module 12)

Specify a complete cryptographic design for a real requirement, including the key lifecycle and the properties you are not providing.

Requirements

- **A stated requirement** with the assets, the adversary, and the properties needed — **confidentiality, integrity, authentication, and freshness treated separately.**
- **Primitive choices with justification**, each one from a reviewed library, with the library's misuse resistance assessed.
- **The complete key lifecycle**: generation, storage, distribution, rotation, revocation, and destruction (Module 10).
- **A named failure mode per component**, and what happens to the rest of the system when it occurs.

- **An explicit statement of what you are not providing** — metadata protection, forward secrecy, deniability, or post-compromise security.
- **And a rotation procedure you have actually executed** against a test deployment.

Done means

- **The four properties treated separately** — **conflating confidentiality with integrity is the failure this project exists to prevent**, and it is graded first.
- **The key lifecycle complete**, including destruction, which is the step always omitted.
- **The rotation procedure executed rather than described** (CSCE 701 Module 07 §3's point).
- **And the not-providing list specific and non-empty. A design that claims to provide everything has not been understood**, and this list is what distinguishes a reviewed design from a hopeful one.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Aumasson — Serious Cryptography , 2nd edition	The primary text, and the right level for this course. Written by a practising cryptographer for engineers who must use the primitives correctly, and it is consistently honest about what is and is not settled. Library copy.
Boneh & Shoup — A Graduate Course in Applied Cryptography (free PDF)	Free in full, and the rigorous companion. Where this course states a property informally, Boneh and Shoup give the definition and the proof. Modules 03 through 07 follow its structure.

Resource	Role
Katz & Lindell — Introduction to Modern Cryptography	The standard text on the definitional approach — why security is defined as a game against an adversary, which is Module 01 §3 and is the idea that makes the rest rigorous. Library copy.
The libsodium documentation (free)	The course's reference implementation, and the embodiment of Module 01's rule. A small, opinionated, misuse-resistant API — read it as a design argument rather than only as documentation.
Cryptographic Right Answers, current edition (free)	A short list of what to use, maintained by practitioners who review real systems. Read it after Module 04 and again after Module 11 — it will read differently.
RFC 8446 (TLS 1.3) and RFC 9106 (Argon2) (free)	Module 08's and Module 09's primary sources. TLS 1.3 is worth reading as an example of a protocol deliberately simplified to remove misuse, which is this course's thesis in standards form.

Tooling

- **A reviewed high-level library, and nothing lower.** `libsodium`, `Google Tink`, or your platform's equivalent — and **Module 01 §4's rule is that the exercises are done with these, not with raw block cipher APIs**, with one deliberate exception noted below.
- **Python with cryptography and pynacl**, which cover every exercise in the course and are both well maintained.
- **One deliberate exception: Module 11's exercises use a low-level API specifically to observe a misuse failing** — on your own test vectors, to build the intuition for why the rule exists. **Nothing from that module goes into a real system.**
- **OpenSSL's command line**, for inspecting certificates and handshakes in Module 08. **Reading a real certificate chain is worth more than a diagram of one.**
- **A hardware security key and a password manager**, used personally — the same recommendation CSCE 701 makes, for the same reason: **the material is more convincing once you have lived with the controls.**
- **And published test vectors** for everything you implement. **A construction that passes the official vectors is correct in a way that one which merely round-trips is not**, and this is the course's single most useful habit.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.